



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



GhostTree Attack Abused Recursive Windows Junctions to Hide Malware

Malware Analysis

Classification	TLP:CLEAR
Published	2026-06-16
Version	1.0
Author	Lost Boys Cyber
Report Type	Malware Analysis

TLP:CLEAR | Lost Boys Cyber | GhostTree Attack Abused Recursive Windows Junctions to Hide Malware - Malware Analysis

Published: 2026-06-16 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Sample Metadata
3. Source Review & Confidence
4. Initial Triage and Delivery Assessment
5. Static Analysis
 - 5.1. File and Build Characteristics
 - 5.2. Strings, Imports, and Embedded Artefacts
 - 5.3. Static Configuration Clues
6. Code Analysis
 - 6.1. Reverse-Engineering Method
 - 6.2. Functions and Logic of Interest
 - 6.3. Anti-Analysis, Evasion, and Obfuscation
7. Dynamic Analysis
 - 7.1. Execution Overview
 - 7.2. Filesystem, Registry, Service, and Task Changes
 - 7.3. Network and Command-and-Control Behaviour
 - 7.4. Observed Capability Summary
8. Process Tree and Execution Chain
9. Persistence and System Modification
10. Network and Infrastructure Analysis
11. Indicators of Compromise
12. MITRE ATT&CK Mapping
13. Detection Engineering
 - 13.1. Detection Logic Summary
 - 13.2. Detection Content
 - 13.3. Detection Validation and Blind Spots
14. Threat Hunting
 - 14.1. Hunt Hypotheses
 - 14.2. Hunt Queries
 - 14.3. Triage Guidance for Hunt Hits
15. Triage and Incident Response
16. Validation and Emulation
17. Recommendations
18. References and Refresh Notes

GhostTree Attack Abused Recursive Windows Junctions to Hide Malware

1. Executive Summary

GhostTree is a Windows file-system evasion technique disclosed by Varonis Threat Labs and later reported by BleepingComputer. The technique abuses NTFS junctions, a legitimate Windows reparse-point feature, to create recursive directory structures that resolve to the same parent folder through a very large number of valid paths. When a security product performs a naive recursive folder scan, the scanner can spend its time walking the junction loop and fail to reach the malicious payload stored in the same folder.

Lost Boys Cyber assesses GhostTree as a malware-hiding and endpoint-evasion tradecraft item rather than a standalone malware family. No executable sample, hash, command-and-control endpoint, or payload family was available in the staged intake. The directly sourced evidence supports the following facts: standard users with write access can create directory junctions; a child junction can point back to its own parent; multiple recursive child junctions create path diversity; and Varonis reported successful testing against Microsoft Defender folder scans before Microsoft patched the issue. This report therefore treats the malware component as hypothetical or operator-provided and focuses on the file-system condition that can conceal it.

Defenders should prioritize visibility into junction creation, recursive reparse-point structures, abnormal scanner behavior, and malware staging in user-writable directories. Endpoint protection should not be the sole control: file-system telemetry, PowerShell and command-line auditing, Sysmon reparse-point events, and scheduled recursive-scan health checks provide better coverage for this class of bypass. Organizations should validate whether their EDR and AV products bound recursion depth, detect cycles, and continue scanning sibling files after encountering reparse points.

Key Finding	Defensive Implication	Priority
NTFS junctions can be created by non-admin users with write access to the parent directory.	User-writable paths can become evasion staging locations without privilege escalation.	High
Recursive junctions can generate vast numbers of valid paths resolving to the same payload.	Recursive scanners can hang, degrade, or miss sibling files if they do not detect cycles.	High
Varonis reported Defender folder-scan evasion and subsequent Microsoft patching.	Patch currency and product-specific validation are required; do not assume endpoint default behavior is sufficient.	High
No public sample hash or payload family accompanied the intake.	Incident response should hunt for the behavior and staging condition, not a fixed IOC list.	Medium

2. Sample Metadata

Field	Value
Family	GhostTree technique; no discrete malware family confirmed
Malware Type	File-system evasion / malware-hiding technique
Platform	Microsoft Windows / NTFS
Primary Sample SHA256	Not available in public reporting
Additional Hashes	Not available
File Type / Format	NTFS directory junctions / reparse points plus optional payload

		executable
Delivery Theme		Post-delivery local staging in a user-writable directory; delivery vector not disclosed
First Seen / Reported		Varonis disclosure published 2026-05-19 and updated 2026-05-20; BleepingComputer coverage published 2026-06-16
Analyst Confidence		Medium overall; high for junction mechanics, medium for real-world exploitation scope
Primary Objective		Hide or delay scanning of malware by trapping recursive folder scans
Evidence Gap	Impact on Analysis	Handling
No malware binary or hash in intake	Static and code analysis cannot validate a real payload	Report separates technique analysis from sample-backed malware claims
No victim telemetry	Execution chain, persistence, and process tree are assessed from expected behavior	Hunts and detections are behavior-based
No vendor-specific patch identifier in source	Exact fixed Defender build cannot be asserted	Recommend product-version validation and vendor confirmation

3. Source Review & Confidence

Source	Contribution	Confidence
Varonis Threat Labs, “GhostTree: Unveiling Path Manipulation Techniques to Bypass Windows Security”	Primary technical explanation of NTFS junction abuse, GhostBranch/GhostTree path construction, path-count estimate, Defender scan test, and Microsoft response	High for mechanics; medium for cross-product generalization
BleepingComputer, “GhostTree Attack Abused Recursive Windows Junctions to Hide Malware”	Secondary media intake source summarizing the same technique and defensive risk	Medium; article content could not be extracted through the local toolchain, but search result metadata and task source metadata corroborate topic/date
Lost Boys Cyber local analysis	Source review, ATT&CK mapping, detection engineering, and hunt logic derived from the public technique description	Medium; no sample execution was performed

Directly observed evidence in the accessible source is limited to the Varonis write-up. The report states that any user can create NTFS junctions with write access, that a junction can point back to its parent, that multiple junctions create binary-tree-like path expansion, and that Varonis tested the technique against Windows Defender folder scans. Lost Boys Cyber did not receive the original proof-of-concept artifacts or a malware payload.

High-confidence assessment: GhostTree should be handled as a practical evasion primitive for Windows malware staging. Lower-confidence inference: adversaries may incorporate this technique into loaders, ransomware staging, or hands-on-keyboard intrusion playbooks; public reporting did not attribute active exploitation to a named actor or malware family.

4. Initial Triage and Delivery Assessment

The intake item was routed as malware analysis because the technique is explicitly framed as a way to hide malware from endpoint scanning. However, GhostTree is not itself a loader, stealer, RAT, or ransomware binary. It is a file-system manipulation that an attacker can apply after obtaining write access to a directory.

Variant / Artefact	Delivery Role	Notes
GhostBranch	Single recursive junction loop	A child junction points back to its parent, creating many valid paths to the same executable.
GhostTree	Multiple recursive child junctions	Two or more child junctions branch at each level, creating a far larger path space.
Payload executable	Hidden or delayed-scan target	Public source used examples such as Program.exe / 1.exe; no real malware hash disclosed.
Scanner process	Defensive target	Recursive folder scans may hang or never reach sibling files if cycle handling is weak.

Expected attacker prerequisites are modest: a foothold, script execution or command-line access, and write access to the staging directory. Administrator rights are not required for the basic junction creation described in the source. Likely staging locations include user profile directories, shared folders with permissive write access, developer workspaces, temporary folders, and compromised file shares.

5. Static Analysis

5.1. File and Build Characteristics

No malware file was available for static analysis. The static artifact is the NTFS reparse-point layout created through normal Windows commands.

Static Signal	Evidence	Analyst Value
Directory junction object	<code>`mklink /J <link> <target>`</code> creates an NTFS junction	Detectable through file-system metadata and command-line telemetry
Parent-recursive target	Child path points back to parent directory	Strong indicator of loop creation and scanner-evasion intent
Multiple sibling junctions	<code>`Child1`</code> and <code>`Child2`</code> both target the same parent	Converts linear loop into path-space explosion
Payload co-location	Same parent contains executable or script payload	Scanner must avoid loops while still scanning sibling files
User-writable location	Junction creation only requires write access	Expands exposure beyond admin-only directories

5.2. Strings, Imports, and Embedded Artefacts

Because no binary sample was provided, there are no recovered imports, strings, resources, compile timestamps, packers, or embedded configurations. For incident response, analysts should instead collect:

- command lines containing ``mklink /J``, ``fsutil reparsepoint``, ``New-Item -ItemType Junction``, or suspicious reparse-point manipulation;
- USN journal, Sysmon, or EDR file events showing junction creation under user-writable directories;
- sibling payload files in the same parent directory as the recursive junctions;
- scanner logs showing long-running scans, path-depth exhaustion, or access errors under the same tree.

5.3. Static Configuration Clues

GhostTree has no malware configuration in the traditional sense. Configuration is represented by the junction topology:

Configuration Element	Example	Defensive Meaning
Link path	`C:\Parent\Child1`	Created object to inspect for reparse-point metadata
Target path	`C:\Parent`	Parent-recursive target indicates loop condition
Branch count	Two or more child links	Determines path diversity and scan workload
Payload path	`C:\Parent\1.exe` or equivalent	File that may be missed by recursive scan
Maximum path behavior	Legacy 260-character limit or long-path behavior	Influences how many path permutations are reachable

6. Code Analysis

6.1. Reverse-Engineering Method

No executable sample was available for Ghidra, radare2, or sandbox-assisted reverse engineering. Analysis is therefore based on the operating-system behavior documented by Varonis and on defensive telemetry modeling. The relevant “code path” is the attacker’s junction creation command and the scanner’s recursive directory-walk behavior.

6.2. Functions and Logic of Interest

Function / Routine	Evidence	Why It Matters
Junction creation	`mklink /J C:\Parent\Child C:\Parent`	Establishes a logical loop without requiring administrator privileges.
Multi-branch loop creation	`mklink /J C:\Parent\Child1 C:\Parent`; `mklink /J C:\Parent\Child2 C:\Parent`	Creates a branching path graph rather than a simple loop.
Recursive scanner traversal	Varonis reported `dir` recursion and EDR-style folder scans can fail to complete	Scanner cycle detection and reparse-point handling become security controls.
Payload sibling placement	Malicious files remain in the same parent while scanner follows the junction tree	The evasion works by distracting traversal rather than hiding bytes inside the junction.

6.3. Anti-Analysis, Evasion, and Obfuscation

GhostTree is a structural anti-scanning technique. It does not require encryption, packing, API hashing, process injection, or complex code obfuscation. Its strength is that all generated paths are valid from the operating-system perspective and can resolve to the same underlying file. With two branch names and a maximum depth around 126 levels in the source example, the possible node count is approximately 2^{126} , or 85,070,591,730,234,615,865,843,651,857,942,052,864 possible paths. This value should be treated as a path-space illustration, not as evidence that every tool will enumerate every path.

7. Dynamic Analysis

7.1. Execution Overview

No sandbox execution was performed because no sample or proof-of-concept package was provided. A safe emulation would create a test-only directory, add recursive junctions, place a harmless marker file as the “payload,” and validate how endpoint tools handle scanning the parent folder. Do not run this test on production endpoints without guardrails because recursive scans can consume time and resources.

7.2. Filesystem, Registry, Service, and Task Changes

Expected filesystem changes are limited to junction objects and any payload files the attacker stores in the parent. Public reporting did not describe registry persistence, service creation, scheduled tasks, WMI persistence, or durable autorun changes. Those may appear in a real intrusion if the concealed payload has separate persistence logic, but they are not inherent to GhostTree.

Artefact	Expected Change	Confidence
NTFS reparse point	Junction created under attacker-writable parent	High
Payload file	Executable/script/document stored as sibling of junctions	Medium; technique-driven assumption
AV/EDR scan state	Long-running or stalled scan under parent directory	Medium; product-specific behavior
Registry/services/tasks	Not inherent to GhostTree	Low unless confirmed by payload telemetry

7.3. Network and Command-and-Control Behaviour

GhostTree itself has no network protocol, beacon, domain, IP address, TLS profile, or command-and-control infrastructure. Any network behavior belongs to the payload hidden near the junction structure. Incident response teams should correlate GhostTree-like filesystem activity with subsequent outbound connections from binaries located in the same parent directory.

7.4. Observed Capability Summary

Capability	Supporting Dynamic Evidence
Create recursive file-system paths	Varonis source shows parent-pointing NTFS junction commands.
Expand path diversity through branching	Varonis source shows Child1/Child2 style loop construction and 2^{126} path-space calculation.
Delay or defeat recursive scanning	Varonis source reports testing against Windows Defender folder scans.
Conceal unrelated malware payloads	Technique places malware in the parent while scanner follows recursive paths; payload not publicly provided.

8. Process Tree and Execution Chain

The expected process tree is short for manual or scripted deployment. Actual process names may vary depending on whether the actor uses CMD, PowerShell, a custom loader, or a remote management tool.

Step	Parent Process	Child / Action	Why It Matters
1	Initial foothold or operator shell	`cmd.exe` or `powershell.exe` launched in user context	Establishes local execution context.
2	`cmd.exe` / `powershell.exe`	`mklink /J` or junction-creation API call	Creates the recursive NTFS reparse point.
3	Same shell or payload dropper	Writes payload into parent directory	Places malware where recursive scanners may not inspect promptly.

4	AV/EDR scanner	Recursive folder scan under parent path	Defensive process may hang, loop, or skip sibling payloads if cycle detection is weak.
5	Payload process	Executes from parent or one of the generated paths	Execution path may be unusually deep, repetitive, or inconsistent across telemetry.

Representative process lineage for command-line creation:

explorer.exe / remote_shell.exe / management_agent.exe
└─ cmd.exe /c mklink /J C:\Parent\Child1 C:\Parent
└─ cmd.exe /c mklink /J C:\Parent\Child2 C:\Parent
└─ payload.exe (optional; not observed in public source)

9. Persistence and System Modification

GhostTree should not be confused with persistence. Recursive junctions can remain on disk and continue affecting scans, but they do not automatically launch code after reboot. Persistence must be assessed separately from the payload.

Modification Type	Artefact	Evidence	Confidence
File-system evasion	Parent-recursive junction	Public source command examples and explanation	High
Payload storage	Executable in parent folder	Technique description; no sample available	Medium
Startup persistence	Run key, service, scheduled task, WMI event	Not reported for GhostTree	Low
Defense interference	Scanner stall or degraded folder scan	Varonis Defender testing claim	Medium-High

Response teams should delete malicious junctions only after preserving metadata, path listings, and the associated payload. Blind deletion can destroy evidence of operator intent and prevent reproduction of the scanner failure.

10. Network and Infrastructure Analysis

No stable network infrastructure is associated with GhostTree. The only “infrastructure” is local file-system topology.

IOC / Infrastructure	Type	Role	Stability / Confidence
Recursive NTFS junction	Local file-system artefact	Scanner-evasion structure	High for technique; path names vary
Parent directory containing payload and junctions	Local staging path	Stores hidden or delayed-scan payload	Medium; depends on intrusion
External C2	Domain / IP / URL	Not part of GhostTree itself	Not available

If a real incident includes GhostTree artefacts, analysts should pivot from the parent directory to process execution, network events, and user/session history. The hidden payload may still expose traditional network IOCs even though the hiding technique does not.

11. Indicators of Compromise

This case has few durable IOCs. Treat the following as indicators of behavior and condition rather than fixed strings.

IOC Type	Example / Count	Source	Operational Notes
----------	-----------------	--------	-------------------

Command line	`mklink /J <child> <parent>` where target is an ancestor of link path	Varonis / derived detection logic	Strong when target resolves to parent or ancestor.
PowerShell action	`New-Item -ItemType Junction` targeting an ancestor path	Derived detection logic	Less common in normal enterprise workflows.
File-system metadata	NTFS reparse point / junction under user-writable directory	Technique behavior	Requires endpoint file metadata visibility.
Path pattern	Repeated directory names or alternating branch names in very deep paths	Technique behavior	Useful for hunting suspicious process/file paths.
Hash/domain/IP	None published	Source limitation	Do not fabricate sample IOCs.

Suggested machine-readable sidecar handling: keep `iocs.csv` intentionally minimal and document that no hashes, domains, or IPs were published. Behavioral detections should live in the detection and hunt sections rather than in IOC-only workflows.

12. MITRE ATT&CK Mapping

Technique	Name	Evidence
T1027	Obfuscated Files or Information	Recursive junctions obscure scanner traversal and can hide a payload from recursive inspection.
T1564.001	Hide Artifacts: Hidden Files and Directories	The payload is not necessarily hidden by attributes, but the directory structure hides it from weak scanning workflows.
T1070.004	Indicator Removal on Host: File Deletion	Not inherent; include only if actors remove junctions or payloads after use in a real case.
T1204.002	User Execution: Malicious File	Possible when the concealed payload later executes; not confirmed in public reporting.
T1059.003	Command and Scripting Interpreter: Windows Command Shell	`mklink /J` examples use CMD syntax.
T1059.001	Command and Scripting Interpreter: PowerShell	Relevant if attackers create junctions via PowerShell or .NET filesystem APIs.
T1006	Direct Volume Access	Not observed; include as a related validation consideration only if tooling bypasses filesystem APIs.

13. Detection Engineering

13.1. Detection Logic Summary

Signal	Telemetry Source	Analytic Goal	Tuning Notes
`mklink /J` creating a link whose target is an ancestor path	Windows process command line, Sysmon Event ID 1, EDR process telemetry	Catch simple operator or script use of CMD junction creation	Legitimate admin scripts may create junctions; ancestor-target condition is the strongest discriminator.

Reparse point / junction creation in user-writable directories	Sysmon FileCreate, EDR file metadata, filesystem audit	Detect non-admin creation of suspicious filesystem topology	Tune known developer tools and package managers that use junctions.
Deep repetitive or alternating path execution	Process, image load, file open telemetry	Surface payloads launched through recursive paths	Tune backup, build, and test harnesses that create deep paths.
Long-running AV/EDR scans under a specific directory	EDR health telemetry, Defender operational logs, scan duration metrics	Identify scanner hang conditions	Requires product-specific operational logs.

13.2. Detection Content

title: Suspicious Recursive NTFS Junction Creation With Mklink id: 807d9b42-8dfb-4e74-a2a4-ghosttree-mklink status: experimental description: Detects mklink junction creation that may be used to build GhostTree-style recursive directory structures. author: Lost Boys Cyber date: 2026-06-17 references: - https://www.varonis.com/blog/ghosttree-ntfs-trick - https://www.bleepingcomputer.com/news/security/ghosttree-attack-abused-recursive-windows-junctions-to-hide-malware/ logsource: product: windows category: process_creation detection: selection_img: Image endswith: '\cmd.exe' selection_cmd: CommandLine contains all: - 'mklink' - ' /J' condition: selection_img and selection_cmd fields: - UtcTime - User - Image - ParentImage - CommandLine - CurrentDirectory falsepositives: - Developer tooling, package managers, backup products, or administrative scripts that legitimately create junctions. level: medium	title: Execution From Very Deep Or Repetitive Windows Path id: d09ae51d-fb9c-4a18-bbd3-ghosttree-deep-path-execution status: experimental description: Flags process execution from paths with repeated directory names or excessive path depth, which can indicate junction-loop abuse. author: Lost Boys Cyber date: 2026-06-17 references: - https://www.varonis.com/blog/ghosttree-ntfs-trick logsource: product: windows category: process_creation detection: selection_depth: Image re: '^([A-Za-z]:\\([\\^\\]+\\)){20,}([\\^\\]+\\.exe\$' selection_repeated_child: Image contains:
--	--

```

- '\\Child\\Child\\'
- '\\Child1\\Child1\\'
- '\\Child2\\Child2\\'
- '\\P\\P\\'
- '\\B\\B\\'
condition: selection_depth or selection_repeated_child
fields:
- UtcTime
- User
- ParentImage
- Image
- CommandLine
falsepositives:
- Test harnesses and build systems that intentionally create deep directory structures.
level: medium

// Microsoft Defender for Endpoint: mklink junction creation in user-writable paths
DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName =~ "cmd.exe"
| where ProcessCommandLine has_all ("mklink", "/J")
| extend Cmd = tostring(ProcessCommandLine)
| where Cmd has_any ("\\Users\\", "\\ProgramData\\", "\\Temp\\", "\\AppData\\", "\\Public\\")
| project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName,
    FileName, ProcessCommandLine, FolderPath, ReportId
| order by Timestamp desc

// MDE: process execution from unusually deep or repetitive paths
DeviceProcessEvents
| where Timestamp > ago(30d)
| extend SlashCount = strlen(FolderPath) - strlen(replace_string(FolderPath, "\\", ""))
| where SlashCount >= 20
    or FolderPath has "\\Child\\Child\\"
    or FolderPath has "\\Child1\\Child1\\"
    or FolderPath has "\\Child2\\Child2\\"
    or FolderPath has "\\P\\P\\"
    or FolderPath has "\\B\\B\\"
| project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName,
    FileName, FolderPath, ProcessCommandLine, SHA256
| order by Timestamp desc

index=windows sourcetype=XmlWinEventLog:Microsoft-Windows-Sysmon/Operational EventCode=1
(Image="*\\cmd.exe" CommandLine="*mklink*" CommandLine="* /J*")
| eval lower_cmd=lower(CommandLine)
| where like(lower_cmd, "%\\users\\%") OR like(lower_cmd, "%\\appdata\\%") OR like(lower_cmd, "%\\
\\programdata\\%") OR like(lower_cmd, "%\\temp\\%")
| table _time host user ParentImage Image CommandLine CurrentDirectory

```

13.3. Detection Validation and Blind Spots

Validate with a harmless lab directory and a non-malicious marker file. Confirm that detections trigger on junction creation and that endpoint scans complete or gracefully skip recursive reparsing points while still inspecting sibling files. Blind spots include API-based junction creation without `mklink`, products that do not expose reparsing-point telemetry, benign developer ecosystems that use junctions heavily, and actors who create junctions through a signed management agent or script interpreter.

14. Threat Hunting

14.1. Hunt Hypotheses

- An adversary with user-level access created recursive NTFS junctions in a writable directory to hide a payload from recursive AV/EDR folder scans.
- A payload executed from or near a path containing repeated directory names, excessive nesting, or alternating branch names created by a GhostTree-like structure.
- Endpoint scanner operational logs show abnormal scan duration, repeated path traversal, or incomplete scan results for a directory that also contains executable content.

14.2. Hunt Queries

```
// Hunt 1: suspicious junction creation through common scripting interfaces
let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where ProcessCommandLine has_any ("mklink", "New-Item", "CreateSymbolicLink", "reparsepoint")
| where ProcessCommandLine has_any ("/J", "Junction", "MountPoint", "SetReparsePoint")
| project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName,
    FileName, FolderPath, ProcessCommandLine, SHA256
| order by Timestamp desc

// Hunt 2: executable or script activity in directories with suspicious path depth
let lookback = 30d;
DeviceProcessEvents
| where Timestamp > ago(lookback)
| where FileName endswith ".exe" or FileName endswith ".dll" or FileName endswith ".ps1" or FileName
endswith ".bat" or FileName endswith ".cmd"
| extend PathDepth = strlen(FolderPath) - strlen(replace_string(FolderPath, "\\ ", ""))
| where PathDepth >= 20
    or FolderPath matches regex @"(?i)(\\child1?){2,}"
    or FolderPath matches regex @"(?i)(\\[pb]){4,}"
| project Timestamp, DeviceName, InitiatingProcessAccountName, InitiatingProcessFileName,
    FileName, FolderPath, ProcessCommandLine, SHA256, PathDepth
| order by PathDepth desc, Timestamp desc

// Hunt 3: payload siblings near mklink-created junction directories
let JunctionCreators = DeviceProcessEvents
| where Timestamp > ago(30d)
| where FileName == "cmd.exe" and ProcessCommandLine has_all ("mklink", "/J")
| project DeviceId, DeviceName, JunctionTime=Timestamp, JunctionCmd=ProcessCommandLine,
Account=InitiatingProcessAccountName;
DeviceFileEvents
| where Timestamp > ago(30d)
| where ActionType in ("FileCreated", "FileModified", "FileRenamed")
| where FileName endswith ".exe" or FileName endswith ".dll" or FileName endswith ".ps1" or FileName
endswith ".bat" or FileName endswith ".cmd"
| join kind=innerunique JunctionCreators on DeviceId
| where Timestamp between ((JunctionTime - 2h) .. (JunctionTime + 2h))
| project Timestamp, DeviceName, Account, FolderPath, FileName, SHA256, JunctionTime, JunctionCmd
| order by Timestamp desc

index=windows (sourcetype=XmlWinEventLog:Microsoft-Windows-Sysmon/Operational OR
sourcetype=WinEventLog:Security)
(CommandLine="*mklink* /J*" OR CommandLine="*New-Item*Junction*" OR CommandLine="*reparsepoint*")
| stats values(CommandLine) as command_lines values(ParentImage) as parents values(Image) as images by
host user
| where mvcount(command_lines) > 0
```

14.3. Triage Guidance for Hunt Hits

A meaningful hit combines junction creation with an ancestor-targeting path, a nearby executable/script, or endpoint scan anomalies. Benign lookalikes include developer package managers, build tooling, application compatibility shims, backup products, and storage redirection scripts. Escalate to incident response when the same host shows suspicious payload creation, execution from deep paths, unsigned binaries, credential access, suspicious outbound connections, or multiple junctions created by an unusual user or remote shell.

15. Triage and Incident Response

Priority	Action	Owner	Notes
Critical	Preserve the parent directory, junction metadata, payload files, command history, and EDR telemetry before cleanup.	IR / Forensics	Capture path listings with reparse-point metadata; avoid recursive commands that can hang.
High	Isolate hosts if a payload is	SOC / Endpoint	GhostTree alone is not malware

	present or execution is observed.		execution, but payload co-location changes severity.
High	Validate AV/EDR product behavior and patch level.	Endpoint Engineering	Confirm recursion-cycle protection and sibling-file scanning.
Medium	Remove malicious junctions after evidence capture.	Endpoint Engineering	Use safe non-recursive tooling and confirm parent payloads are scanned.
Medium	Hunt for related execution, persistence, and network activity.	Threat Hunting	Pivot from parent directory, user, process ancestry, and hash.

For evidence collection on Windows, prefer tools that can enumerate reparse points without following them recursively. Record link path, target path, owner, timestamps, parent directory ACLs, and sibling executable hashes. If scanners previously hung, collect product operational logs and scan IDs for vendor escalation.

16. Validation and Emulation

This report is source-backed rather than sample-backed. Lost Boys Cyber did not run a live Windows detonation or reverse-engineering session for this intake. Safe validation can still be performed in an isolated Windows VM:

```
mkdir C:\GhostTreeLab
copy C:\Windows\System32\notepad.exe C:\GhostTreeLab\marker.exe
mklink /J C:\GhostTreeLab\Child1 C:\GhostTreeLab
mklink /J C:\GhostTreeLab\Child2 C:\GhostTreeLab
```

Validation objectives:

Test	Expected Result	Pass Criteria
Junction detection	Telemetry captures `mklink /J` and reparse-point creation	SOC detection fires with correct host/user/path.
Scanner behavior	Endpoint product avoids infinite recursion	Scan completes and inspects `marker.exe`.
Hunt coverage	Deep/repetitive path queries surface lab execution if marker runs through recursive path	Hunt returns lab host and path.
Cleanup	Junctions removed without deleting parent payload evidence	Parent folder, logs, and payload hashes remain preserved.

Do not use a malicious payload for validation. The objective is to prove telemetry and scanner resilience, not to test malware execution.

17. Recommendations

Priority	Owner	Action	Rationale
Critical	Endpoint Engineering	Confirm Microsoft Defender and any third-party EDR/AV are updated to versions that handle recursive junctions safely.	Varonis reported Defender folder-scan evasion and subsequent patching; product behavior must be validated locally.
High	SOC Detection Engineering	Deploy detections for `mklink /J`, PowerShell junction creation, and	The technique has weak fixed IOCs but strong behavioral

		deep/repetitive execution paths.	signals.
High	Threat Hunting	Run a 30-day retrospective hunt for junction creation in user-writable directories and payload siblings.	Detects prior staging and validates whether the technique is already present.
High	Windows Platform Engineering	Audit writable directories and shared folders where standard users can create reparse points.	GhostTree only requires write access; reducing writable staging locations lowers abuse surface.
Medium	IR / Forensics	Add reparse-point handling to collection playbooks.	Recursive commands can hang or miss evidence if responders do not account for junction loops.
Medium	Security Architecture	Require scanner cycle detection and sibling-file inspection as endpoint protection acceptance criteria.	Prevents future bypasses based on filesystem graph manipulation.
Medium	User Awareness / Admin Training	Document that junctions and symlinks can be security-relevant artifacts, not just storage conveniences.	Improves triage quality for helpdesk, desktop, and SOC teams.

18. References and Refresh Notes

- [1] Varonis Threat Labs, “GhostTree: Unveiling Path Manipulation Techniques to Bypass Windows Security,” published 2026-05-19, updated 2026-05-20: <https://www.varonis.com/blog/ghosttree-ntfs-trick>
- [2] BleepingComputer, “GhostTree Attack Abused Recursive Windows Junctions to Hide Malware,” published 2026-06-16: <https://www.bleepingcomputer.com/news/security/ghosttree-attack-abused-recursive-windows-junctions-to-hide-malware/>
- [3] Microsoft documentation, NTFS reparse points and junction/symbolic-link behavior should be consulted during product-specific validation.

Refresh notes: update this report if Microsoft or EDR vendors publish CVE identifiers, fixed-version matrices, product advisories, or real-world exploitation telemetry. If a future incident includes a payload, produce a sample-backed malware appendix with hashes, static/code/dynamic analysis, process tree, network infrastructure, and payload-specific detections.